Báo cáo

1. Trước khi nạp dữ liệu

- Thiết lập môi trường db
- Phân quyền: finova_loader, finova_dbt, finova_app (được dùng để check rls ở tầng mart).
- Schema ctrl hệ thống:
    - source_system: Hệ thống nguồn dữ liệu đổ về.
    - batch_log: log thông tin nạp dữ liệu vào hệ thống nguồn.
    - load_reject: Thông tin các gói bị từ chối.
    - schema_drift: Ghi thông tin về cột mới (nếu có)
    - batch_run: Thông tin về lô chạy
    - error_log: Bảng ghi log tập trung
    - guard_block: Thông tin về lô bị chặn
    - guard_resolution: Thông tin về xử lý lô chặn
    - reject_count: Số dòng bị chặn theo lô
- Hợp đồng nguồn:
    - 1 dạng cam kết đầu ra của các model, dbt buộc phải tuân thủ nếu có sự sai lệch thì sẽ báo lỗi khi build
    - Có thể dùng để mapping khi nguồn đổ về lưu tên cột bằng tiếng việt.
- Thứ tự nạp vào finova:
    - Có cột arrival_ts ở trong bảng manifest.csv là thứ tự đến của từng lô
    - Hệ thống nạp mô phỏng theo ngày D => Chỉ nạp vào những lô có available_ts <= cutoff_ts(D). Ở đây, available_ts là thời điểm file có mặt vậy lý tại inbound, khác với thời điểm hệ thống nguồn xuất file là commit_ts, thông thường available_ts và commit_ts sẽ bằng nhau

2. Nạp vào tầng landing

- Input:
    - contract/*.yml; manifest.csv, 6 luồng file vật lý
- Các điều kiện kiểm tra:
    - Kiểm tra trùng mã lô: batch_id đã ở trong ctrl.batch_log => Không nạp vào.
    - Kiểm tra tồn tại vật lý: Kiểm tra xem trong inbound có tồn tại file không. Đồng thời, tự tính sha256 của file để so sánh với sha256 trong manifest, nếu lệch => Loại.
    - Đọc dữ liệu thô và kiểm tra số dòng: Tiến hành đọc dữ liệu và giữ nguyên tình trạng tài liệu (trừ dòng trắng không đọc), với excel sẽ

transfer tiêu đề thành tên cột tiếng anh phù hợp. Ngoài ra, tiến hành đếm số dòng thực tế, nếu sai khác => Từ chối cả lô.
    - Kiểm tra dữ liệu lặp: So sánh content_hash với các lô có trạng thái 'LOADED' ở trong batch_log.
    - Kiểm tra cấu trúc dữ liệu: Thừa/thiếu cột so với hợp đồng sẽ gán cờ tương ứng, vẫn nạp vào landing
    - Kiểm tra 5 cột hệ thống và lọc dòng hỏng: commit_ts → ingested_ts; tính event_ts, nếu không tính được → Loại; dòng hợp lệ → Tính record hash, nạp đủ 5 cột hệ thống.
- Xử lý bên trong:
    - Chuẩn bị và khởi tạo cấu trúc bảng:
        - Khởi tạo bảng với 8 cột kỹ thuật (5 hệ thống + source_row_no + 2 cột hỗ trợ cho excel).
        - Xử lý trùng tên cột (event_ts trong finova_app_events)
    - Transfer vào landing:
        - Truyền nguyên text vào không tiến hành can thiệp sửa chữa, chỉ đổi rỗng thành null.
        - Xử lý đặc thù cho excel.
    - Tính toán 8 cột kỹ thuật:
        - source_id: mã hệ thống nguồn.
        - batch_id: mã lô từ manifest
        - ingested_ts: commit_ts
        - event_ts: không có cột thời gian → commit_ts; với finova_app_events → giờ thiết bị or server_ts nếu quá 5p; nếu vẫn none → Loại.
        - record_hash: tính mã băm cho 1 dòng nguồn
        - source_row_no: thứ tự dòng dữ liệu trong file
        - excel_row_no: Dòng vật lý trong excel
        - source_col_map: Bản đồ tọa độ trong excel
    - Nạp vào db: landing.<source_table> → ctrl.batch_log → ctrl.load_reject → ctrl.schema_drift → commit (Toàn bộ luồng này được đặt trong 1 transaction).
- Output: 6 bảng dữ liệu trong landing + 4 bảng ctrl trong ctrl.

3. Tầng staging

- Tầng staging sẽ lấy output của tầng landing để xử lý. Toàn bộ stg là view, mục đích của tầng stg là nhằm chuẩn hóa, tính hashkey, hashdiff bộ dữ liệu.

- Xử lý bên trong:
    - Chuẩn hóa: Sử dụng macro để tiền xử lý trước khi tiến hành ép kiểu để tránh các lỗi có thể gây sập view (31/02/2026)
    - Tính khóa: hk để định danh, hd để xác nhận sự thay đổi.
        - hub:
            - regrp_hk(hash('FLEX', 'grpid')).
            - broker_hk(hash('FLEX', 'broker_id')).
            - account_hk(hash('FLEX', 'afacctno'))
            - reaflnk_hk(hash('FLEX', 'autoid'))
        - link, nhl:
            - link_regrp_hk(hash('FLEX', 'grpid', 'parentid'))
            - link_broker_regrpd_hk(hash('FLEX', 'broker_id', 'grpid'))
            - link_reaflnk_hk(hash('FLEX', 'autoid', 'broker_id', 'afacctno')
            - nhl_broker_event_hk(hash('FINOVA', 'event_id').
    - Khử trùng, bắt xung đột và vòng đệ quy:
        - Gom nhóm theo ingested_ts và khóa nghiệp vụ.
        - Trùng → Lấy dòng đầu tiên, các dòng sau bị loại
        - Xung đột → Loại tất cả
        - Dò cây tổ chức mọi độ dài với cte recursive. Nếu phát hiện vòng lặp → Loại
    - Mask dữ liệu theo yêu cầu (mobile, email,..)
    - Xử lý dòng lỗi (loại/giữ) theo quy ước đề.
        - Loại: 4 case mục 2.5 (KEY_NULL, KEY_FORMAT, ALLCODE_UNKNOWN, xung đột/lặp khóa thời điểm nạp) và 3 case của riêng cap-mg (FRDATE>TODATE, CYCLE_DETECTED, lỗi công thức excel)
        - Giữ: Với các cột phụ (on_valid: null_and_count)
        - 1 dòng có thể mắc nhiều lỗi → Phân định ưu tiên lỗi → FORMULA_ERROR > KEY_NULL > KEY_FORMAT > ALLCODE_UNKNOWN > ràng buộc nghiệp vụ/cấu trúc > CONFLICT_IN_BATCH > DUPLICATE_ROW) → Đảm bảo chỉ 1 lỗi ghi vào _rejected.
- Output: 6 view dữ liệu và 6 view dữ liệu bị reject.

4. Tầng raw vault

- Input: Sử dụng nguồn view từ tầng staging

- T-Guard: Được sử dụng để chặn những lô phát lại, đứng trước staging nhưng vì staging là view không liên quan đến lo_ts và hi_ts → Cơ chế nhằm bảo vệ raw vault
- Cách tính lo_ts và hi_ts:
    - lo_ts = hi_ts của lần chạy thành công gần nhất, lần đầu or full refresh là -infinity.
    - max_log_id = max(log_id) ở trong ctrl.batch_log.
    - hi_ts = max(ingested_ts) của các lô LOADED có log_id <= max_log_id.
- Cơ chế xử lý lô muộn và lô phát lại:
    - Lô muộn: Về muộn sau thời điểm cutoff(D) thì vẫn sẽ được nạp vào vault tuy nhiên sẽ chuyển vào thời điểm cutoff(D) tiếp theo
    - Lô phát lại: Lô này sẽ có thời điểm ingested_ts < lo_ts (do available_ts > commit_ts) và log_id > max_log_id → Chặn không được nạp vào vault.
- Xử lý:
    - Hub: Có những thực thể nào?
        - Hợp mọi nguồn mang broker_hk
        - Lấy key ở nguồn sớm nhất
        - Khi key không tồn tại → Thêm key.
    - Link: Thực thể nào từng đi với thực thể nào
        - Tương tự với cách lấy key của tầng hub, tuy nhiên sẽ loại bỏ dòng null (HO không có cha → kh lấy)
    - NHL:
        - Dùng để ghi lại các sự kiện bất biến (không sửa được các hành động ở quá khứ). Tiến hành lưu khóa ngoại của các bảng liên quan và thuộc tính đồng thời lưu chèn thêm hk nếu chưa có. Chỉ insert, không update.
    - SAT: Nạp lịch sử thuộc tính của hub
        - Loại bỏ các dòng trùng lặp trong cùng 1 thời điểm.
        - So sánh giữa các lô trong cùng cửa sổ để đánh số thứ tự.
        - Đối chiếu dòng đầu cửa sổ với vault → Tìm phiên bản liền kề ngay phía trước → Khác thì thêm vào (Kể cả gây ra vòng lặp A → B → A)
    - SAT-DAY: Nạp sat theo ngày nghiệp vụ → Phục vụ bảng nav
        - Thêm business_date vào hashdiff.

        - nav không được đưa vào hashdiff để so sánh với nhau, chỉ được so sánh với bản đính chính (cùng business_date).
    - ESAT: link trả lời câu hỏi thực thể này từng đi với thực thể nào nhưng không thể giải quyết được câu hỏi hôm này thực thể này thuộc thực thể nào → Tạo ra esat nhằm giải quyết (bật/tắt sổ từng link)
        - Sử dụng driving_key để đánh dấu
    - STS: Dùng để theo dõi xóa ngầm trên bản đầy đủ
- Output: 4 hub + 4 link & nhl + 5 sat + 2 esat + 4 sts
- Vấn đề khác:
    - Tái sử dụng mã môi giới: Chỉ hash(nguồn, broker_id) để hợp được 4 nguồn với nhau dễ dàng. Khi cần xác định tại 1 thời điểm là ai sẽ sử dụng tầng bv/gold để xác định.
    - Giải quyết bài toán nạp lại (lũy đẳng): Khi ta nạp dữ liệu vào vault thì sẽ so với dữ liệu liền kề phía trước tuy nhiên nếu ta tiến hành nạp lại thì dữ liệu đầu tiên sẽ so với dữ liệu mới nhất ở trong vault → Nạp vào - điều này là sai gây dư thừa → Khi nạp dữ liệu sẽ so 2 điều kiện
        - Đk1: Khác hk và ingested_ts (nếu trùng đồng nghĩa đã có sẵn)
        - Đk2: Chỉ so với dòng dữ liệu có thời gian liền trước dòng dữ liệu đang được nạp vào
    - Sử dụng sts thay cho is_deleted để tránh phải tạo thêm dòng mới không có trong staging (khi xóa thì staging sẽ không có dòng này nhưng sat muốn biểu thị thì bắt buộc phải thêm dòng). Đồng thời sts đọc cả rejected nhằm đảm bảo phân biệt được dữ liệu bị từ chối hay bị xóa.

5. Tầng asof

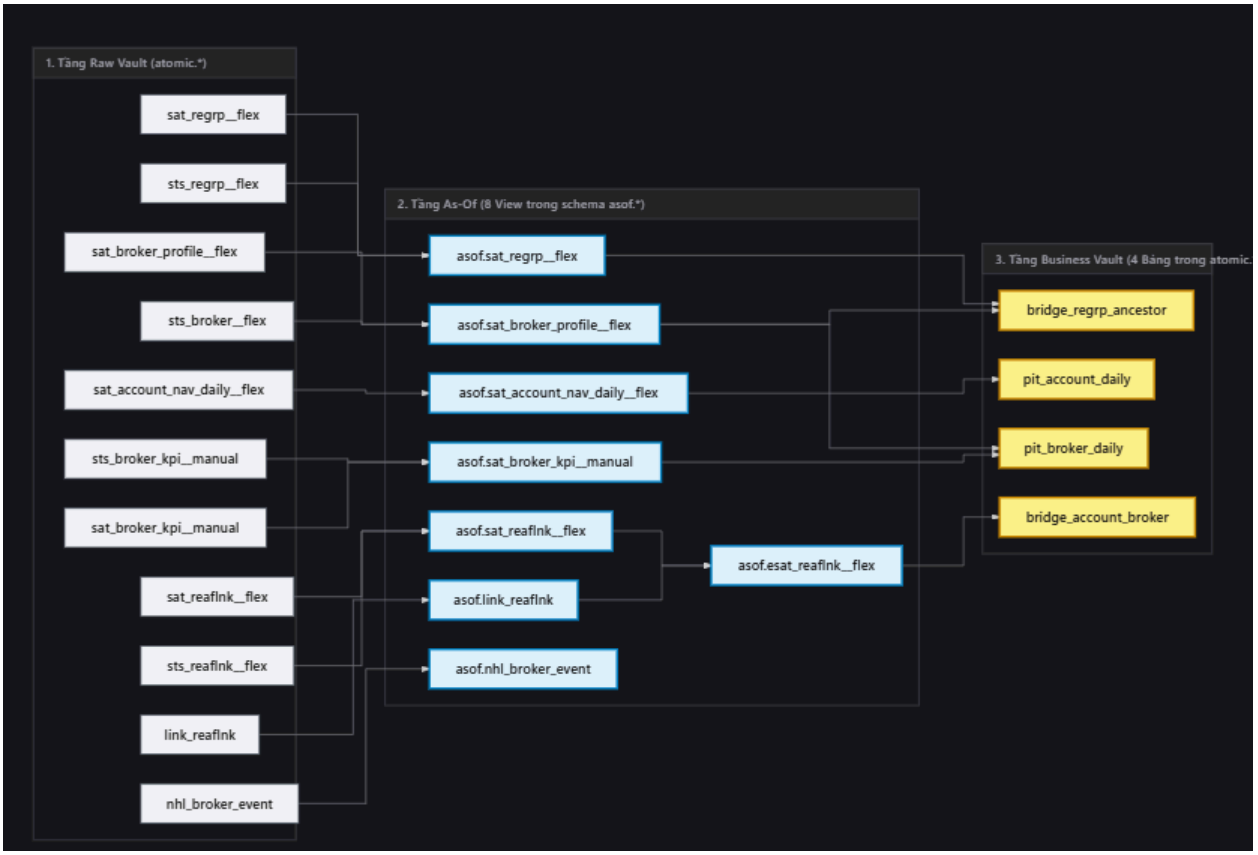
- Sử dụng biến phiên Posgres (GUC)
    - asof.as_of_date: Đọc phiên current_setting('finova.asof_date') trả về kiểu date
    - asof.cutoff(): Đọc biến phiên current_setting('finova.asof_cutoff') trả về kiểu timestampz.

=> Giúp view không phải tạo lại mỗi ngày.

- Nếu nhúng cứng ngày D thông qua var('as_of_date') vào câu lệnh tạo view → Mỗi lần tra cứu ngày khác sẽ phải tạo lại view → Đọc qua hàm asof.cutoff() giúp view hoàn toàn tĩnh → Muốn đọc chỉ cần set_config('finova.asof_cutoff) trên connection → View trả dữ liệu đúng mốc. Ngoài ra khắc phục được lỗi quên đặt giờ.

Được chia thành 4 nhóm dựa trên đặc điểm của từng bảng ở tầng raw vault:

- Dành cho link & nhl: Link và NHL chỉ thêm mới chứ không ghi đè lên nhau → VIew chỉ cần lấy mọi dòng đã nạp vào hệ thống tính đến thời điểm cutoff.
- Dành cho các bảng sat (trừ sat của nav):
    - Lấy phiên bản mới nhất tính đến thời điểm cutoff
    - Lọc bỏ các thực thể bị xóa ngầm trong sts
- Dành cho sat của nav: Đặc điểm của nav hơi khác so với các bảng sat khác khi mà mỗi ngày giá trị nav là 1 khác, độc lập với nhau nên không thể dùng chung như các sat khác.
    - Lọc cả 2 chiều thời gian là ingested_ts <= asof.cutoff và business_date <= asof.as_of_date → Lấy phiên bản mới nhất.
- View cho hiệu lực phân công: Cần tách thành 1 view riêng vì với hiệu lực phân công cần giải bài toán nghiệp vụ nạp lùi hiệu lực → Cần tới 2 trục thời gian để trả lời cho 2 câu hỏi là hôm đó view biết gì và hiện view về ngày hôm đó.



6. Tầng business vault.

- Cơ chế nạp: Trước khi insert vào bảng thì dbt tự động kích hoạt 2 prehook:

- asof_session: Gọi set_config để đặt giá trị cho finova.asof_date và finova. asof_cutoff cho phiên hiện tại để các view asof ở dưới đọc được.
- asof_day_delete: Nhằm xóa sạch dữ liệu tại ngày D → Tránh nạp trùng dữ liệu.

- Thông tin về 4 bảng:
  - pit_broker_daily:
    - Grain: 1 dòng môi giới tại 1 ngày giao dịch
    - Mục đích: Cung cấp broker_hk và ingested_ts để lấy thông tin chính xác của 1 broker về profile và kpi tại đúng 1 thời điểm cần xét

    → Gold chỉ cần join theo cặp broker_hk và ingested_ts → Ra kết quả mong muốn

    - Logic xử lý:
      - Tìm giờ nạp hồ sơ mới nhất → Thu được broker_hk và ingested_ts của profile
      - Tương tự với kpi
      - Outer join 2 phần lại với nhau → Thu được bảng pit của broker.
  - pit_account_daily:
    - Grain: 1 tiểu khoản có phát sinh dữ liệu nav tại 1 ngày giao dịch
    - Mục đích: Để lấy được thông tin chính xác nav tại ngày D
    - Logic xử lý:
      - Đọc từ view của asof_sat_account_nav_daily_flex → Lấy phiên bản ingested_ts mới nhất.
      - Lọc giữ lại dòng có business_date = D → Xuất mã tiểu khoản (account_hk) kèm giờ nạp (ingested_ts).
  - bridge_account_broker:
    - Grain: 1 tiểu khoản tại 1 ngày giao dịch, và đúng duy nhất 1 môi giới được tính kpi trên tiểu khoản đó.
    - Mục đích: Xử lý các sự kiện bị chồng lấn → 1 tiểu khoản trong 1 ngày chỉ có 1 dòng.
    - Logic xử lý:
      - Lấy mọi phân công đang có hiệu lực tại ngày D.
      - Góm nhóm theo account_hk, đếm môi giới/tiểu khoản → Môi giới/tiểu khoản > 1 → Đánh dấu.

            - Lấy theo thứ tự ưu tiên: frdate mới hơn > ingested_ts muộn hơn > autoid lớn hơn → Lấy kết quả đầu tiên.
    - bridge_regrp_ancestor:
        - Grain: Đơn vị cấp dưới thuộc về 1 đơn vị cấp trên và cách nhau mấy bậc.
        - Mục đích: Làm phẳng mối quan hệ.
        - Logic xử lý:
            - CTE recursive để truy vấn mối quan hệ từ con lên cha.
            - Đỉnh cuối là 'HO' → Nhánh bình thường. Ngược lại, là nhánh mồ côi xử lý tiếp.
            - Kiểm tra có nhân viên môi giới không?
                - Không → Loại ra bridge.
                - Có → Tạo 1 nút unknown nối từ cha của nhánh tìm được lên 'HO'.

7. Tầng gold.

- Nạp vật lý theo kiểu incremental cho từng giao dịch như bv.
- Trước khi tính ngày D, prehook tự động gọi asof_session và asof_day_delete.
- Không dùng materialized='table' → Khi tạo lại bảng thì RLS theo bảng sẽ bị biến mất.
- Thông tin các bảng:
    - dim_broker_org_daily:
        - Grain: 1 người mang mã môi giới tại 1 ngày giao dịch.
        - Mục đích: Tại thời điểm D, broker_id đang thuộc về ai, hồ sơ môi giới và trạng thái làm việc.
        - Logic xử lý:
            - Dựa vào ingested_ts của profile trong bảng pit → Lấy đúng phiên bản hồ sơ môi giới tại ngày D.
            - Ráp đường dẫn cây từ gốc xuống nhánh.
            - Xác định trạng thái làm việc.
            - Nối đường dẫn đơn vị với broker_id.
    - dim_account_care_daily:
        - Grain: 1 Tiểu khoản tại 1 ngày giao dịch.
        - Mục đích: Tại ngày D, tiểu khoản đang do ai chăm sóc.
        - Logic xử lý:

        - Gom toàn bộ tiểu khoản hệ thống biết tại ngày D (hợp tiểu khoản trong reaflnk và phát sinh số dư trong nav).
        - Left join với bridge_account_broker → Lấy người chăm sóc.
        - Gán sang người cụ thể và kiểm tra nghỉ việc.
- sec_visibility_daily:
    - Grain: 1 cặp (người xem, tiểu khoản) có được nhìn thấy nhau tại ngày D không.
    - Mục đích: Đảm bảo chỉ môi giới và những người liên quan được xem thông tin tiểu khoản thuộc mình chăm sóc.
    - Logic xử lý:
        - Lấy danh sách quyền từ bảng ref_org_roles → So sánh với các tiểu khoản đang được chăm sóc trong ngày D.
- met_broker_daily:
    - Grain: Bộ chỉ số hiệu quả kinh doanh của 1 người tại 1 ngày giao dịch.
    - Mục đích: Tạo 1 báo cáo trung tâm nhằm đánh giá hiệu suất môi giới.
    - Cách tính chỉ số:

Tên cột	Ý nghĩa & Công thức
n_accounts	Đếm số tiểu khoản mà 1 môi giới đang chăm sóc.
book_nav & n_accounts_without_nav	Tổng tiền nav tại D của các tiểu khoản mà 1 môi giới đang chăm sóc. Nếu nav tại ngày D đang bị khuyết thì cộng 0 và ghi số lượng vào cột n_accounts_without_nav
interactions_30d	Đếm số sự kiện tương tác hợp lệ trong 30 ngày tính đến ngày D.
cover30_num, cover30_den, cover30	Tỷ lệ độ phủ chăm sóc khách hàng trong 30 ngày, cover30_den là mẫu số = n_accounts, cover30_num là tử số = Tổng tiểu khoản có ít nhất 1 tương tác với môi giới.
call_qtd	Đếm số sự kiện loại CALL hợp lệ từ đầu quý của ngày D tới D.
kpi_target_calls, kpi_batch_id, kpi_ingested_ts	Chỉ tiêu số cuộc gọi cả quý giao cho môi giới

- dq_mg_daily:
    - Grain: 1 lỗi được phát hiện trong 1 lô nạp của 1 bảng nguồn thuộc báo cáo ngày D.
    - Mục đích: Xác định xem ngày nào có nạp dữ liệu vào có lỗi và lỗi ở đâu.
    - Logic xử lý:
        - Xác định lô nào thuộc ngày giao dịch D (thông qua ingested_ts và business_date với case chưa vào được landing).
        - Thu thập lỗi:
            - Lỗi loại dòng.
            - Các cảnh báo nghiệp vụ nhưng không loại dòng.
            - Lỗi nạp không thành công vào landing.
        - Gom nhóm và join để lấy từ điển lỗi.
-